Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

Report on Laboratory Work No. 4

The xv6 Shell from the Outside

Operating Systems

Done by: st. gr. FAF-241, Cațț Chiril

Checked by:
asist. univ. Reițman Patricia

Chișinău – 2026

Contents

Purpose of the Work **1**

Environment check **1**

1 Part 1: Updating the xv6 sleep program **1**

2 Part 2: Bash and xv6 shell comparison **3**

3 Part 3: Who expands arguments? **5**

4 Part 4: Reading `user/sh.c` **7**

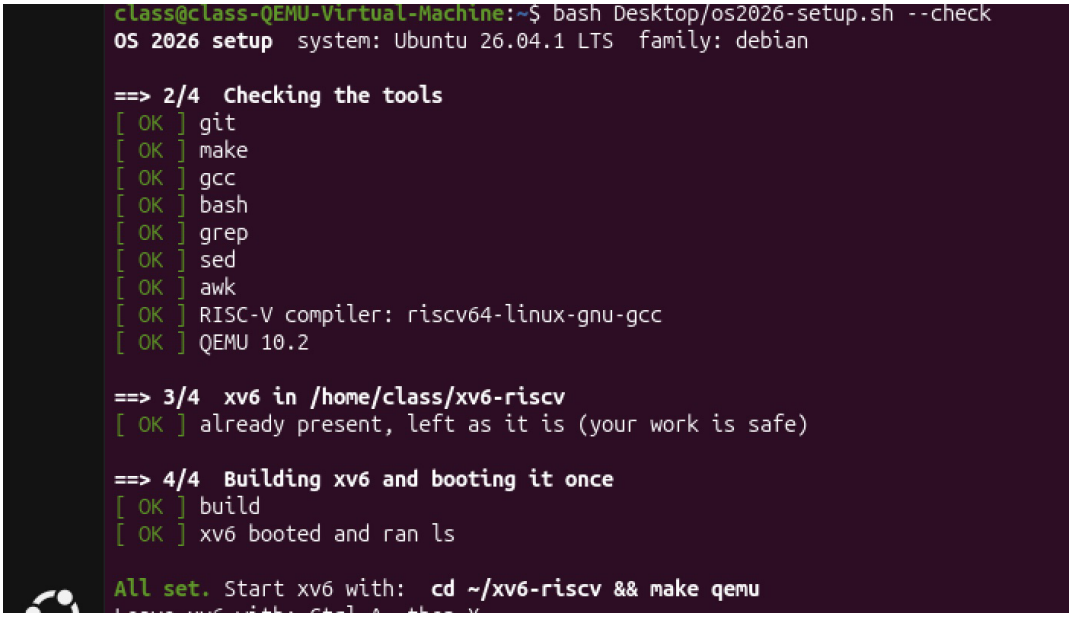
Conclusions **8**

Purpose of the Work

The purpose of this laboratory is to compare the Bash and xv6 command interpreters through direct experiments, update the xv6 `sleep` utility for the current system call interface, write argument inspection programs for both environments, and trace the relevant behavior to `user/sh.c`.

Environment check

I read the Linux setup guide and ran `bash ~/Desktop/os2026-setup.sh -check` in the Ubuntu guest. It confirmed the required tools, found the existing `~/xv6-riscv` tree, built xv6, booted it, and ran `ls`. The final status was `All set`. The RISC-V compiler and QEMU were present. No reinstall was needed.



Successful setup check in the Ubuntu guest.

1 Part 1: Updating the xv6 sleep program

`user/user.h` declares `int pause(int);` on line 25. Accordingly, the xv6 `sleep` program converts its single argument to ticks and calls `pause(atoi(argv[1]))`. For a missing argument it prints usage and exits with status 1. The Makefile includes `$U/_sleep` in `UPROGS`; the guest build succeeded.

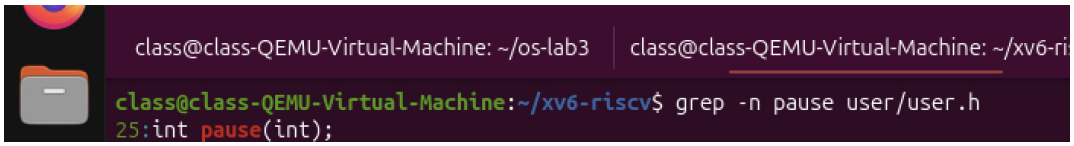


Figure 1.1 – The `pause` declaration in `user/user.h`.

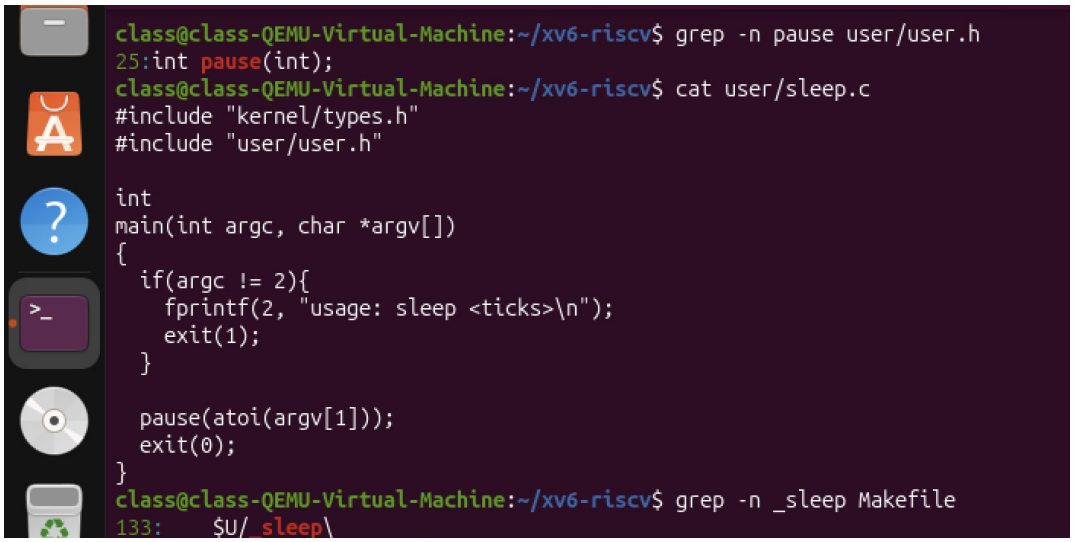


Figure 1.2 – Updated `sleep.c` and its Makefile entry in the guest.

```
$ sleep 20
$ sleep
usage: sleep <ticks>
```

The first command returned to the prompt after its pause; the second took the error path. Both ran in the xv6 shell after `make qemu`.

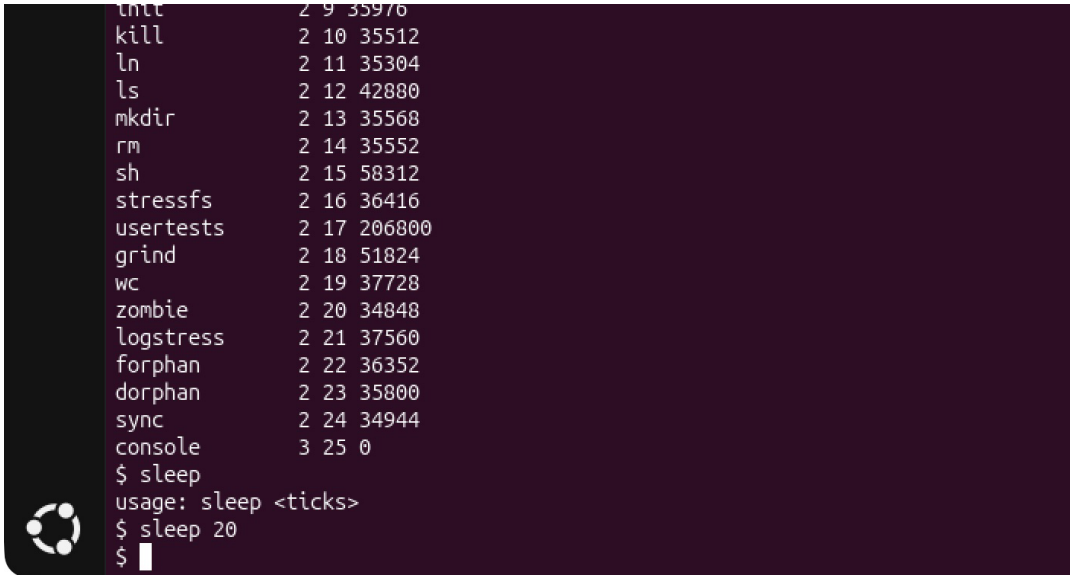


Figure 1.3 – The successful pause and missing-argument test in xv6.

2 Part 2: Bash and xv6 shell comparison

I ran the commands in Bash in a Linux test directory containing `README` and in the xv6 root directory. The Linux test directory also contained small `.c` files used in Part 3. The outputs below record the relevant text; xv6's `ls` additionally prints file type, inode, and size.

Command / feature	Bash result	xv6 result	Explanation
`echo hello > f ; cat < f`	`hello`	`hello`	Both support redirection and sequencing.
`ls | grep READ`	`README`	`README` with metadata	Both create a pipe.
`(echo a; echo b) | wc`	`2 2 4`	`2 2 4`	Grouping, list, and pipe work.
`echo one ; echo two`	`one`, then `two`	Same	Semicolon executes a list.
`echo bg &`	`bg` (background job)	`bg`; prompt can interleave	Both launch a background command.
`ls *`	Lists matched names, including `README`	`ls: cannot open *`	Bash expands the wildcard; xv6 passes it literally.
`echo $HOME`	`/home/class`	Literal `$HOME`	Bash expands variables.
`echo "a b"`	`a b`	Literal `"a b"`	Bash removes quotes and keeps one argument; xv6 does neither.
`mkdir d ; cd d ; ls ..`	Parent names displayed	`exec ls failed` after `cd d`	Bash finds `ls` through `PATH`; xv6 searches the current directory.
Up arrow, Tab, Ctrl+R	History and completion	No corresponding line editing	Bash provides interactive editing.

The shared features are execution, input/output redirection, pipelines, command lists, grouping, background execution, and `cd`. Bash adds filename globbing, variable expansion, useful quoting, `PATH` lookup, history, and completion. In the Bash run, `echo $HOME` returned `/home/class`; `echo "a b"` returned `a b`. The guest transcript is preserved in `lab4-results.txt`.

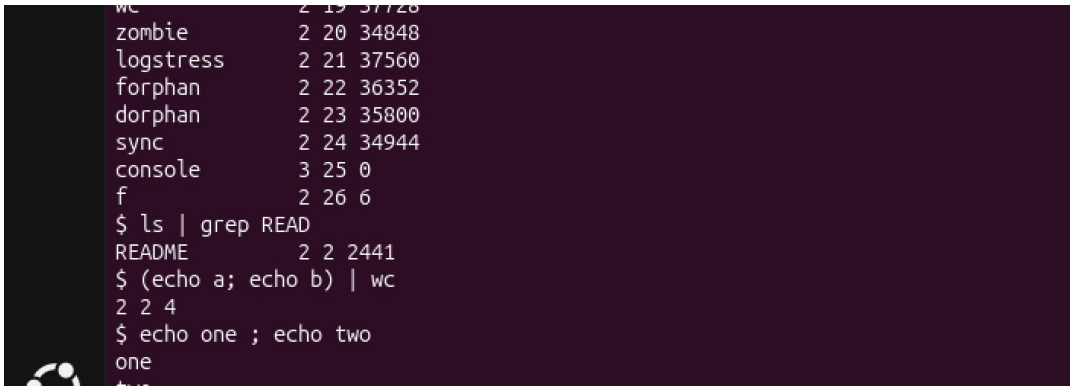


Figure 2.1 – Pipelines and command lists working in xv6.

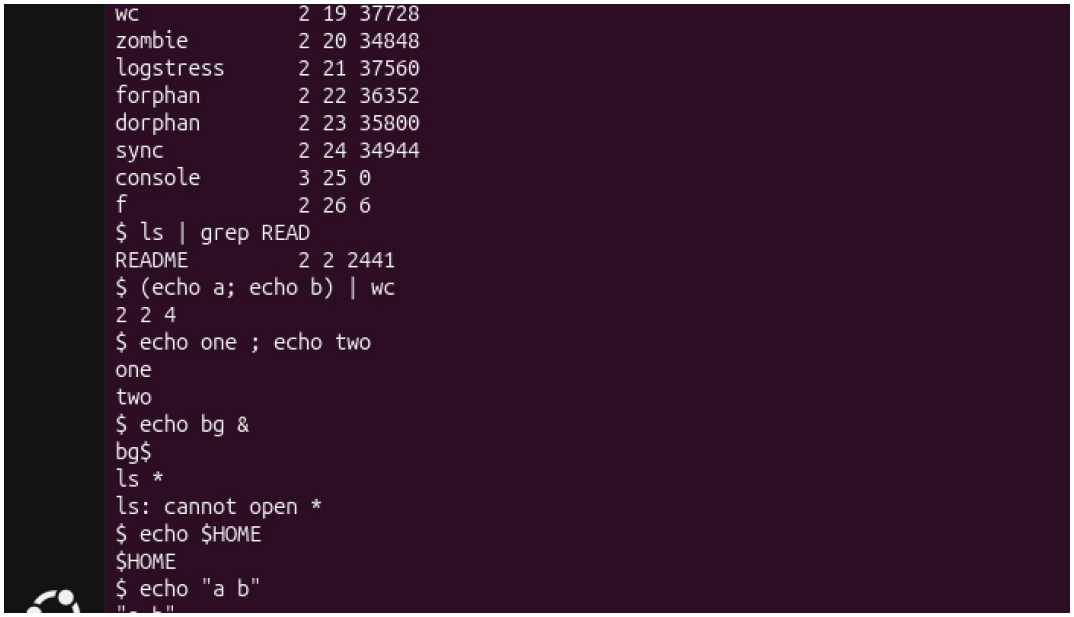


Figure 2.2 – The xv6 shell passes the wildcard, variable name, and quotes literally.

After `cd d`, xv6 attempts to execute `ls` relative to `d`, where that program is absent. The executable is in the parent (root) directory, so `../ls` succeeds. Bash uses its `PATH` to find `ls` after a directory change.

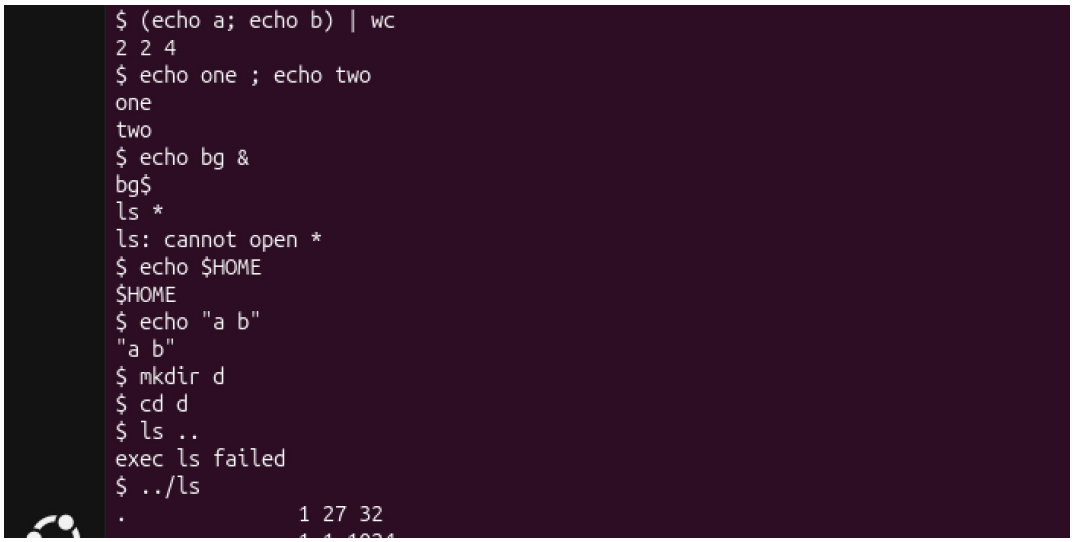


Figure 2.3 – `ls` failing from `d` and `../ls` succeeding.

I would miss history and completion most in daily use because they save repeated typing. A limited single-entry history recall would be the easiest feature to add: it only needs to store and replay the last input line. General quoting or globbing needs parser changes and filename expansion.

3 Part 3: Who expands arguments?

I wrote `args.c` for xv6 and `args_linux.c` for Linux. Each prints `argc` and every `argv` element. The xv6 Makefile includes `$U/_args`. I compiled the Linux version with `gcc -o args args_linux.c`; `make` built the xv6 version.

Listing 1: Core of the two argument inspection programs.

```
printf("argc = %d\n", argc);
for(int i = 0; i < argc; i++)
  printf("argv[%d] = [%s]\n", i, argv[i]);
```

The decisive command was `args *.c "x y"`. The Linux directory contained three C files: `a.c`, `args_linux.c`, and `b.c`. Bash produced:

```
argc = 5
argv[0] = [/home/class/args]
argv[1] = [a.c]
argv[2] = [args_linux.c]
argv[3] = [b.c]
argv[4] = [x y]
```

In xv6 the same line produced:

```
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
```

Thus Bash passed four user arguments, while xv6 passed three. The shell, rather than the program or kernel, expands `*.c` and processes quotes. The program receives only the resulting argument vector.

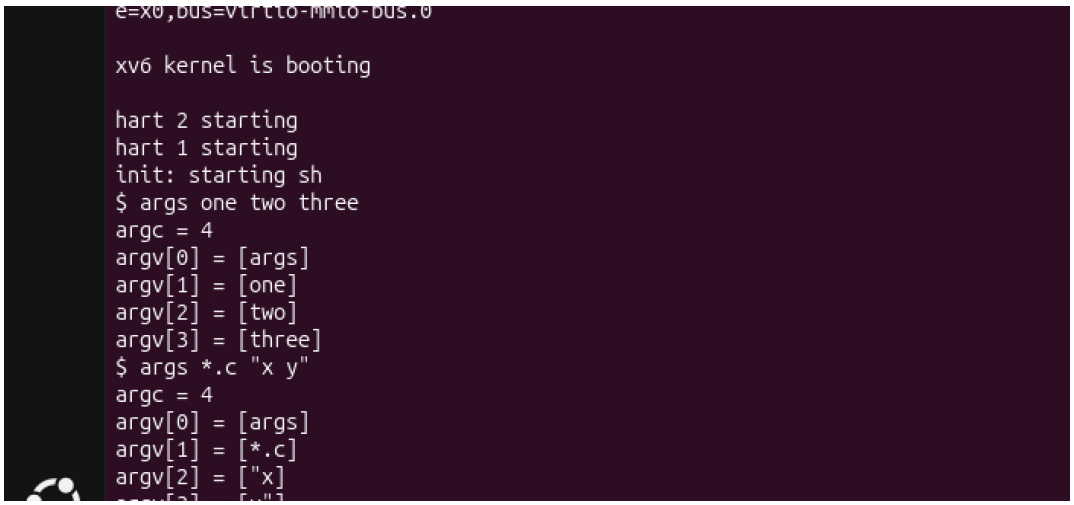


Figure 3.1 – The xv6 argument printer with ordinary words and the wildcard/quotes test.

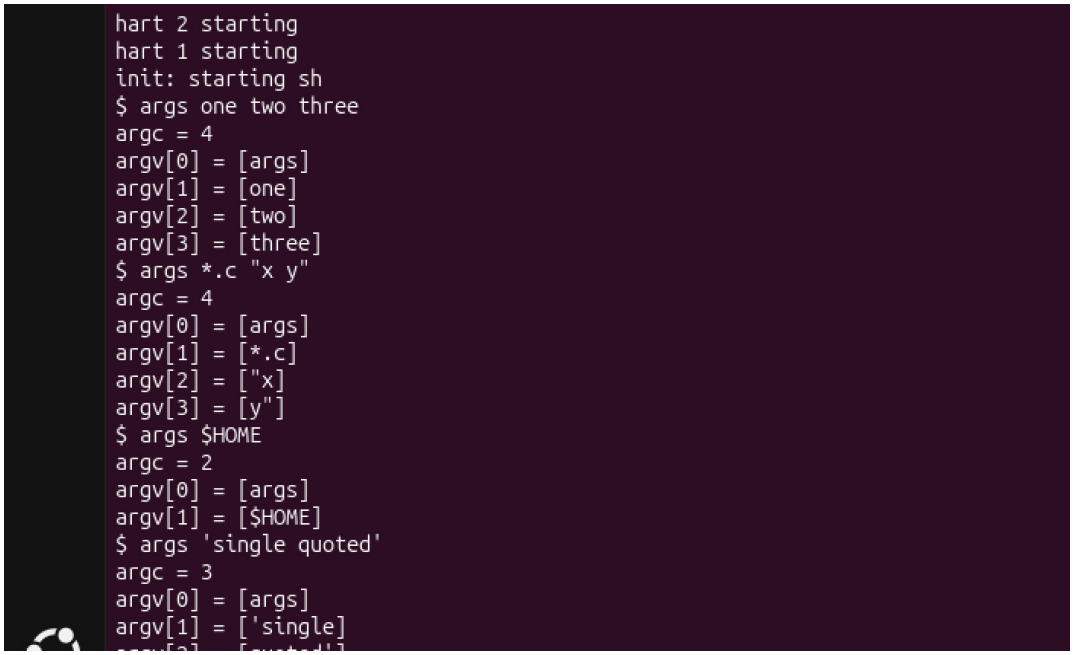


Figure 3.2 – The xv6 argument printer with variables and single quotes.

With `args one two three`, both received `argc = 4`; repeated spaces only separate tokens

and are not passed inside the arguments. Bash expanded `$HOME` to `/home/class` and turned `'single quoted'` into one argument without quotes. xv6 passed `$HOME` literally and split the quoted words into two arguments with quote characters retained.

4 Part 4: Reading `user/sh.c`

The guest's `user/sh.c` has 499 lines. Its command tags are defined near the top and dispatched in `runcmd`:

Tag	Input syntax	Action
`EXEC`	a program name (no operator)	Execute a program.
`REDIR`	`<`, `>`, `»`	Redirect a file descriptor.
`PIPE`	`|`	Connect two commands by pipe.
`LIST`	`;`	Run commands in sequence.
`BACK`	`&`	Run without waiting in the foreground.

```
8:#define EXEC  1       75:  case EXEC:
9:#define REDIR 2       83:  case REDIR:
10:#define PIPE  3       101: case PIPE:
11:#define LIST  4       93:  case LIST:
12:#define BACK  5       125: case BACK:
```

The `cd` check is in the shell's main input loop, around lines 164–172, before the usual fork and command execution. Changing directory must affect the shell process itself. If `cd` ran as a separate child program, only that child's working directory would change; the next prompt would still be in the old directory.

```
char *cmd = buf;
while (*cmd == ' ' || *cmd == '\t')
  cmd++;
if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
  cmd[strlen(cmd) - 1] = 0;
  if (chdir(cmd + 3) < 0)
    fprintf(2, "cannot cd %s\n", cmd + 3);
} else {
  if (fork1() == 0)
    runcmd(parsecmd(cmd));
  wait(0);
}
```

The parser has no wildcard or variable-expansion stage. It treats `*` and `$` as ordinary word

characters. This explains the literal `ls *` failure and the `$HOME` output in Part 2. The shell also lacks Bash-style quote processing, matching the Part 3 argument results.

Conclusions

The environment check, xv6 build, and boot completed successfully. Updating `sleep` to call `pause` made it work with this tree. Direct shell tests showed that xv6 supports the core execution operators but lacks the expansion and interactive conveniences of Bash. The argument printers established that expansion and quote handling occur in the shell before a program starts. Reading `sh.c` connected those observations to its five command types and its special handling of `cd`.